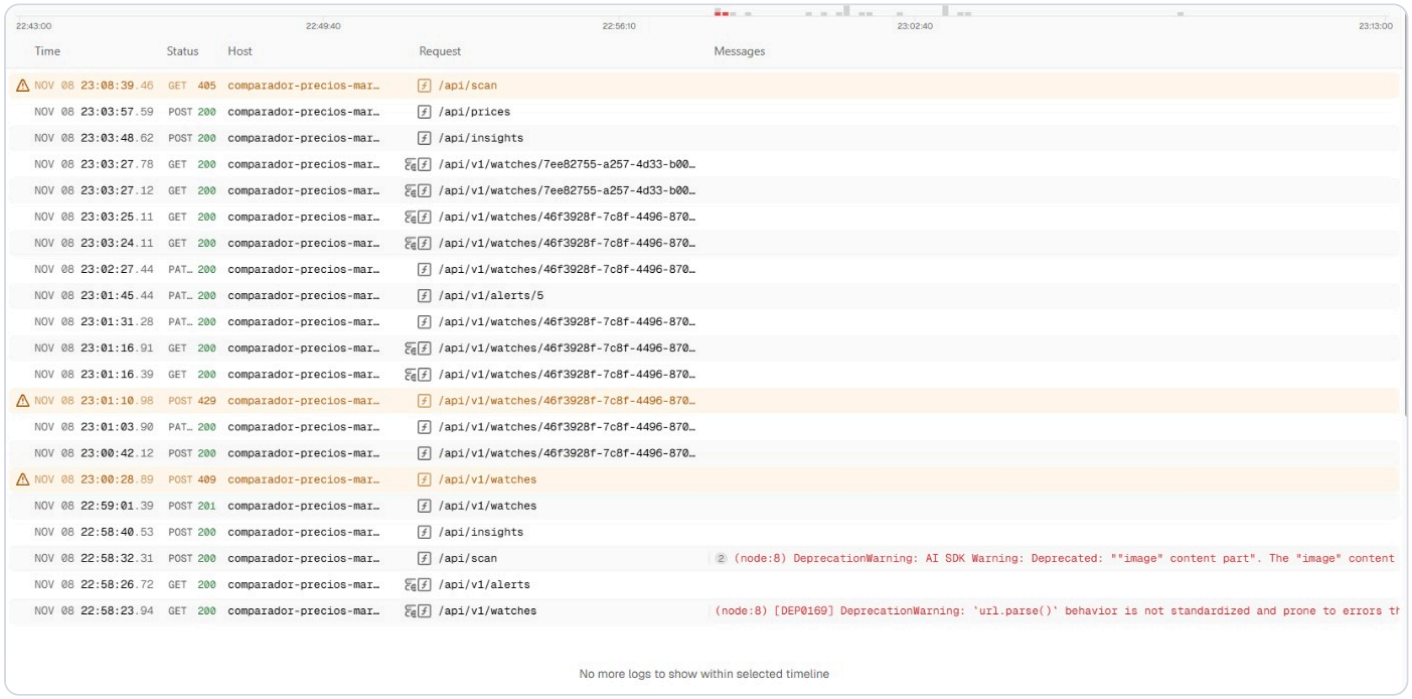
Demostración · cliente Android consumiendo la API propia

Taller 3 · SI806 · Brian Curay, Marco Obispo, Niels Chaves · github.com/MarcOBL012/comparador-precios · prueba del 2026-11-08 contra producción (Vercel + Supabase)

La app Android llama a la API REST propia desplegada en Vercel y los cambios quedan guardados en Postgres (Supabase). La captura de abajo es el registro de peticiones de Vercel (*Request Logs*) durante la prueba: cada fila es una llamada real de la app, con su método, ruta y código HTTP.

Operaciones realizadas (hora Perú)

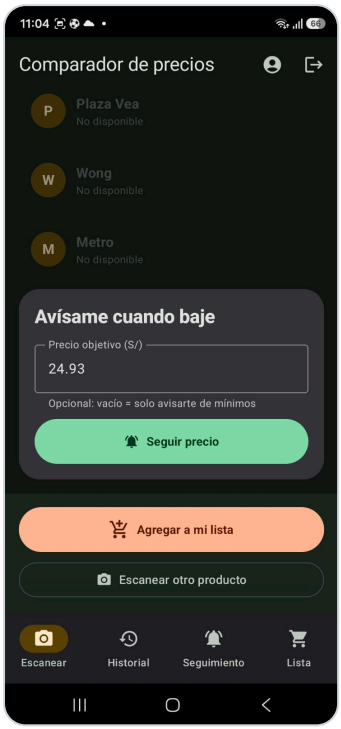
Hora	Petición	Código	Qué se hizo en la app
22:58:23	GET /api/v1/watches	200	Se abre la pestaña Seguimiento: la app carga la lista
22:58:32	POST /api/scan	200	Foto del producto: se identifica y se comparan precios
22:59:01	POST /api/v1/watches	201	«Seguir precio»: se crea el seguimiento (**cambio persistido**)
23:00:28	POST /api/v1/watches	409	Seguir el mismo producto otra vez: `ya_seguido`
23:00:42	POST /api/v1/watches/{id}/check	200	«Revisar ahora»: consulta tiendas y guarda precios (operación de negocio)
23:01:10	POST /api/v1/watches/{id}/check	429	Revisar de nuevo enseguida: regla R3, una revisión cada 15 min
23:01:03 · 23:01:31 · 23:02:27	PATCH /api/v1/watches/{id}	200	Tres actualizaciones: objetivo y pausar/reactivar
23:01:45	PATCH /api/v1/alerts/5	200	Se marca la alerta como leída
23:03:27	GET /api/v1/watches/{id}/history	200	«Ver historial»: mínimo, promedio y máximo



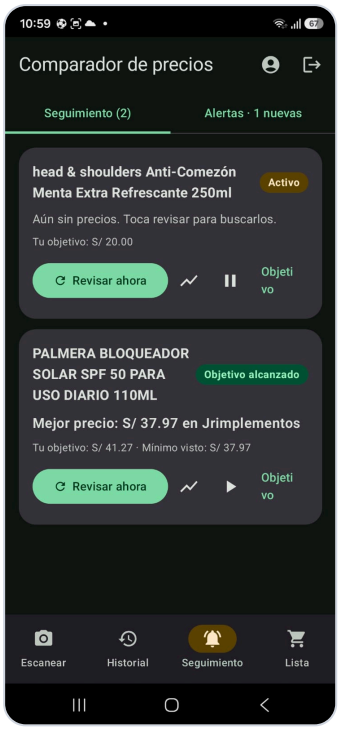
Vercel · Request Logs del proyecto en producción (captura original). La fila naranja `GET 405` en `/api/scan` (23:08) es posterior a la prueba: una consulta con método no permitido.

Pantallas de la app (celular)

Capturas originales del dispositivo durante la misma prueba. Cada pantalla usa un endpoint de la API.



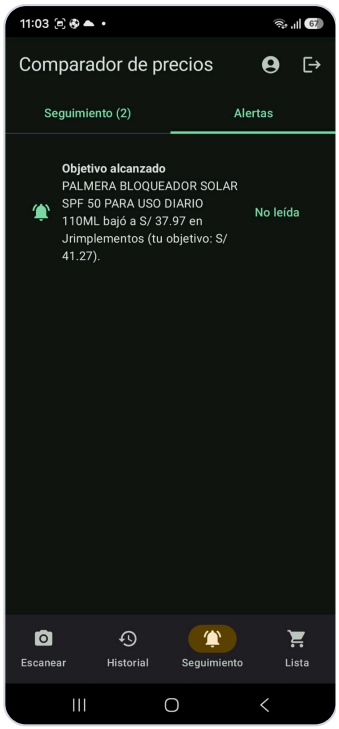
1 · «Avísame cuando baje»
precio objetivo y «Seguir precio» → `POST /watches`



2 · Seguimiento (2)
estado, objetivo y acciones → `GET /watches`



3 · Historial de precios
mínimo, promedio, máximo → `GET /watches/{id}/history`



4 · Alertas
«Objetivo alcanzado» → `GET /alerts`

> Las pantallas 2 y 4 muestran los datos que devuelve la API: el producto con su estado («Activo», «Objetivo alcanzado»), el mejor precio y la alerta generada por la regla de negocio.

Cambio persistido en la base de datos (1/2)

Supabase en producción. Las horas coinciden con las de la página 1: creación 22:59:02, revisión 23:00:44, alerta leída 23:01:45.

Tabla watches (seguimientos) con su producto

Supabase · proyecto DAIS (rama main, PRODUCTION) · consultado 2026-11-08 23:25 (hora Peru)

CONSULTA SQL

```
select p.nombre as producto, p.presentacion, w.target_price as objetivo,
       w.status_code as estado, w.last_checked_at, w.created_at
from watches w join products p on p.id = w.product_id
order by w.created_at desc;
```

RESULTADO

Producto	Present.	Objetivo (S/)	Estado	Última revisión	Creado
Anti-Comezón Menta Extra Refrescante	250ml	22.00	activa	2026-11-08 23:00:44	2026-11-08 22:59:02
BLOQUEADOR SOLAR SPF 50 PARA USO DIARIO	110ML	41.27	alcanzada	2026-11-04 06:22:35	2026-11-03 20:40:35
Agua Mineral de Manantial		1.20	activa	2026-11-08 06:22:36	2026-11-03 20:23:11

QUÉ DEMUESTRA

La primera fila se creó el 2026-11-08 a las 22:59:02 (`POST /watches`) y su última revisión es de las 23:00:44 (`POST /watches/{id}/check`): el cambio quedó persistido. La fila «alcanzada» muestra la transición de estado cuando el precio llegó al objetivo (regla R4).

Resultado real de la consulta SQL (SQL Editor / API de Supabase), sin identificadores de usuario. Generado el 2026-11-08 23:29 (hora Perú) a partir de salida real. Es una representación de datos, no una captura de pantalla de la herramienta.

Tabla alerts (alertas generadas por la regla de negocio)

Supabase · proyecto DAIS (rama main, PRODUCTION) · consultado 2026-11-08 23:25 (hora Peru)

CONSULTA SQL

```
select id, alert_type_code as tipo, created_at as creada, read_at as leida
from alerts order by created_at desc;
```

RESULTADO

Id	Tipo	Creada (Perú)	Leída (Perú)
5	objetivo_alcanzado	2026-11-04 06:22:35	2026-11-08 23:01:45

QUÉ DEMUESTRA

La alerta **objetivo_alcanzado** se creó el 2026-11-04 a las 06:22, en la hora de la revisión automática (regla R4). El campo **read_at** se llenó el 2026-11-08 a las 23:01:45 al marcarla como leída (`PATCH /alerts/{id}`).

Resultado real de la consulta SQL (SQL Editor / API de Supabase), sin identificadores de usuario. Generado el 2026-11-08 23:29 (hora Perú) a partir de salida real. Es una representación de datos, no una captura de pantalla de la herramienta.

Cambio persistido en la base de datos (2/2)

Historial de precios guardado por la revisión y por la revisión automática diaria.

Tabla price_observations (historial de precios)

Supabase · proyecto DAIS (rama main, PRODUCTION) · consultado 2026-11-08 23:25 (hora Peru)

CONSULTA SQL

```
select o.observed_at, s.nombre as tienda, o.precio, o.source_code as fuente, l.titulo
from price_observations o
join store_listings l on l.id = o.listing_id
join stores s on s.id = l.store_id
order by o.observed_at desc limit 12;
```

RESULTADO

Observado	Tienda	Precio (S/)	Fuente	Título
2026-11-08 23:00:44	Falabella	27.70	shopping	Shampoo Head & Shoulders Anti Comezón
2026-11-08 06:22:36	Metro	7.90	directa	Agua Sin Gas Mineral San Mateo Bidón 7L
2026-11-08 06:22:36	Wong	2.10	directa	Agua Sin Gas Mineral San Mateo Botella 600ml
2026-11-08 06:22:36	Plaza Vea	2.00	directa	Agua Mineral SAN MATEO Con Gas Botella 600ml
2026-11-07 06:22:35	Wong	2.10	directa	Agua Sin Gas Mineral San Mateo Botella 600ml
2026-11-07 06:22:35	Metro	7.90	directa	Agua Sin Gas Mineral San Mateo Bidón 7L
2026-11-07 06:22:35	Plaza Vea	2.00	directa	Agua Mineral SAN MATEO Con Gas Botella 600ml
2026-11-06 06:22:43	Wong	2.10	directa	Agua Sin Gas Mineral San Mateo Botella 600ml
2026-11-06 06:22:43	Metro	7.90	directa	Agua Sin Gas Mineral San Mateo Bidón 7L
2026-11-06 06:22:43	Plaza Vea	2.00	directa	Agua Mineral SAN MATEO Con Gas Botella 600ml
2026-11-05 06:22:36	Wong	2.10	directa	Agua Sin Gas Mineral San Mateo Botella 600ml
2026-11-05 06:22:36	Plaza Vea	2.00	directa	Agua Mineral SAN MATEO Con Gas Botella 600ml

QUÉ DEMUESTRA

La primera fila (Falabella, S/ 27.70, fuente «shopping») corresponde a la revisión de las 23:00:44. Las de las 06:22 coinciden con la revisión automática diaria (ver supabase_04).

Resultado real de la consulta SQL (SQL Editor / API de Supabase), sin identificadores de usuario. Generado el 2026-11-08 23:29 (hora Perú) a partir de salida real. Es una representación de datos, no una captura de pantalla de la herramienta.

Revisión automática diaria (cron) funcionando en producción

Supabase · proyecto DAIS (rama main, PRODUCTION) · consultado 2026-11-08 23:25 (hora Peru)

CONSULTA SQL

```
select date_trunc('day', observed_at) as dia, count(*) as observaciones,
       min(observed_at) as primera_hora
from price_observations group by 1 order by 1 desc;
```

RESULTADO

Día	Observaciones	Primera hora (Perú)
2026-11-08	4	06:22:36
2026-11-07	3	06:22:35
2026-11-06	3	06:22:43
2026-11-05	3	06:22:36
2026-11-04	5	06:22:35
2026-11-03	6	20:23:28

QUÉ DEMUESTRA

Cada día del 4 al 8 de noviembre aparecen observaciones a las 06:22 hora Perú (11:22 UTC), a la misma hora todos los días. Coincide con la revisión automática `/api/v1/internal/check-due` programada en vercel.json (`0 11 * * *`, UTC; en el plan gratuito Vercel puede demorarla unos minutos).

Resultado real de la consulta SQL (SQL Editor / API de Supabase), sin identificadores de usuario. Generado el 2026-11-08 23:29 (hora Perú) a partir de salida real. Es una representación de datos, no una captura de pantalla de la herramienta.

Colección de Postman: 26 peticiones, éxitos y fallos

Corrida completa contra la API v1 local (Docker + Postgres), con la colección entregada en `docs/api/postman_collection.json`.

Postman/Newman · Resumen de la colección completa

docs/api/postman_collection.json contra http://localhost:3000/api/v1 (equivalente al Runner de Postman)

RESULTADO

26	33/33	0
peticiones	comprobaciones superadas	fallos

DETALLE POR PETICIÓN

#	Método	Petición	Código	Tests
1	POST	Crear seguimiento	201	3/3
2	POST	Crear repetido → 409	409	2/2
3	POST	Crear con datos inválidos → 400	400	2/2
4	GET	Listar	200	1/1
5	GET	Listar con parámetro inválido → 400	400	1/1
6	GET	Detalle	200	1/1
7	GET	Detalle de id inexistente → 404	404	1/1
8	GET	Detalle con id mal formado → 400	400	1/1
9	PATCH	Cambiar objetivo	200	1/1
10	PATCH	Pausar	200	1/1
11	PATCH	Estado que el usuario no puede pedir → 400	400	1/1
12	POST	Revisar un seguimiento pausado → 409	409	1/1
13	PATCH	Reactivar	200	1/1
14	GET	Sin sesión → 401	401	1/1
15	PUT	Método no permitido → 405	405	1/1
16	POST	Revisar ahora	200	1/1
17	POST	Revisar otra vez enseguida → 429	429	1/1
18	GET	Historial	200	2/2
19	GET	Historial con days inválido → 400	400	1/1
20	GET	Listar alertas	200	2/2
21	PATCH	Marcar alerta como leída	200	1/1
22	PATCH	Marcar alerta con cuerpo inválido → 400	400	1/1
23	GET	Cron sin secreto → 401	401	1/1
24	GET	Cron con secreto	200	2/2
25	DELETE	Eliminar seguimiento	204	1/1
26	DELETE	Eliminar otra vez → 404	404	1/1

Generado el 2026-11-08 23:29 (hora Perú) a partir de salida real. Es una representación de datos, no una captura de pantalla de la herramienta.

Pruebas automáticas y regla de negocio

13 pruebas de integración escriben en Postgres real; la suite completa son 199 pruebas. Abajo, el detalle de la regla R3 (429 con `Retry-After`).

Pruebas de integración contra Postgres real (13)
test/integration/api.pg.test.ts · Vitest

COMANDO

```
TEST_DATABASE_URL=postgresql://postgres:postgres@localhost:54329/comparador npx vitest run
test/integration --reporter=verbose
```

RESULTADO

✓ CRUD completo: crea, lee, lista, edita y borra, persistiendo en la base
✓ normalización: dos personas que siguen el mismo producto comparten UNA fila de producto
✓ un seguimiento repetido responde 409 y uno ajeno 404
✓ check: guarda las ofertas, calcula el resumen y el historial refleja el cambio persistido
✓ R4: llegar al objetivo crea la alerta, pasa a alcanzada y genera eventos de dominio
✓ alertas: se listan, se filtran por no leídas y se marcan como leídas
✓ R3: una segunda revisión antes de 15 minutos responde 429 con Retry-After; después de 15, funciona
✓ un seguimiento pausado no se puede revisar (409)
✓ si las tiendas fallan responde 502 y no guarda nada
✓ R6: un precio atípico se descarta y no contamina el historial
✓ atomicidad: si algo falla a mitad de la revisión, no queda ningún dato a medias
✓ cron: exige el secreto y revisa los seguimientos atrasados, aislando los errores
✓ borrar un seguimiento arrastra sus alertas (cascada) pero conserva el historial de precios del producto

RESUMEN

13/13
pruebas pasaron

NOTA

Los mensajes «timeout» y «violates check constraint» que Vitest imprime en esta corrida son de las pruebas **«si las tiendas fallan responde 502»** y **«atomicidad»**: fuerzan el fallo a propósito y la prueba verifica que no quede ningún dato a medias.

Generado el 2026-11-08 23:29 (hora Perú) a partir de salida real. Es una representación de datos, no una captura de pantalla de la herramienta.

Postman/Newman · 17. Revisar otra vez enseguida → 429
Carpeta: 2. Precios y alertas (lógica de negocio)

PETICIÓN

POST `http://localhost:3000/api/v1/watches/ebb261cd-f170-4b95-83c9-7c4d9c4d87a2/check`

ENCABEZADOS ENVIADOS

```
X-Dev-User: postman
User-Agent: PostmanRuntime/7.39.1
Accept: */*
Cache-Control: no-cache
Postman-Token: bf6c0bd6-9305-4984-a123-03404e8141a8
Host: localhost:3000
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Content-Length: 0
```

RESPUESTA

429 Too Many Requests 11 ms · 109 bytes

ENCABEZADOS DE RESPUESTA

```
Retry-After: 900
Content-Type: application/json; charset=utf-8
Transfer-Encoding: chunked
```

CUERPO DE LA RESPUESTA (JSON)

```
{
  "error": "Revisaste este producto hace poco. Vuelve a intentarlo en 15 min.",
  "codigo": "demasiadas_consultas"
}
```

COMPROBACIONES DE LA COLECCIÓN

✓ **429 con Retry-After (o 502)**

Generado el 2026-11-08 23:29 (hora Perú) a partir de salida real. Es una representación de datos, no una captura de pantalla de la herramienta. Servidor: API v1 local (Docker + Postgres).

> Las tablas de Supabase, Postman y pruebas son representaciones generadas a partir de salida real (cada imagen lo indica en su pie). Las pantallas del celular y el registro de Vercel son capturas originales.